



American University of Sharjah

College of Engineering

Department of Computer Science and Engineering

IMDb Movie Database Management System

CMP 320 - Database Systems

Fall 2025

Mohammed Nour - b00098185

Mouaz Kassouma - b00097195

Yazan Alkazimi - b00096484

Professor: Dr. Mohammad Hamza

26/11/2025

Table of Contents

1. Project Description.....	2
2. ER Diagram.....	2
3. Relational Model.....	3
3.1. ER to Relational Mapping.....	3
3.2. Relational Schema.....	3
3.3. Entity Analysis.....	3
3.4. Relationship Analysis.....	6
4. Implementation.....	9
4.1. DDL Statements.....	9
4.2. Data Population.....	15
5. Query Demonstration.....	30
5.1. Query 1: Find all titles with their ratings and genres.....	30
5.2. Query 2: Count the number of titles directed by each director.....	30
5.3. Query 3: Find actors and the characters they played in specific titles.....	31
5.4. Query 4: Average rating and total votes by genre.....	31
6. Conclusion.....	33
7. References.....	34

1. Project Description

This database system manages comprehensive information about movies, TV shows, and the entertainment industry using IMDb datasets. The system tracks titles (movies, TV series, episodes), people involved in productions (actors, directors, writers), ratings, and various metadata about productions, including alternate titles across different regions and languages. The database also incorporates production company information to track the business aspects of film and television production.

2. ER Diagram

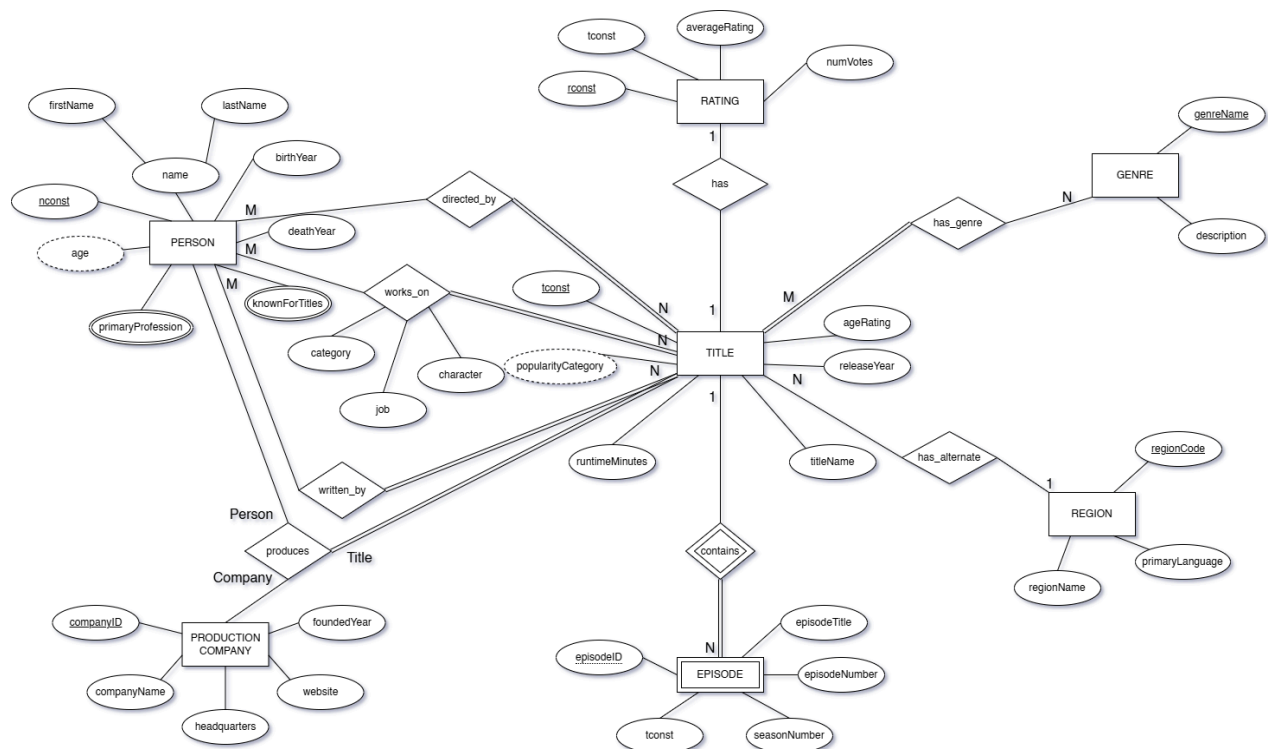


Figure 1: Complete ER Diagram showing entities, attributes, and relationships

3. Relational Model

3.1. ER to Relational Mapping

To start off our mapping, we mapped the regular entities, then the weak entity Episode with a foreign key referencing Title. Afterwards, we mapped our only 1:1 relationship, has_Rating, and made it its own relation as both sides had partial participation. The next step was mapping the binary 1:N, for which we took the primary key of the relation marked with 1 will be used as a foreign key in the relationship marked with an N. Moreover, the binary M:N relationships were mapped afterwards, which included many of the relationships in this diagram. The relationships were represented as their own relations with the foreign key of the two relations connecting it being used as a composite primary key (e.g. Works_On). Then we had to map our multivalued attributes, of which we had two: primaryProfession and knownForTitles. These were represented as their own relations as well, with a foreign key pointing back to the original relation they were a part of. Finally, we move on to mapping the N-ary relationship Produces, which took the three foreign keys of the relations connected to it and made them a composite primary key in its own relation.

3.2. Relational Schema

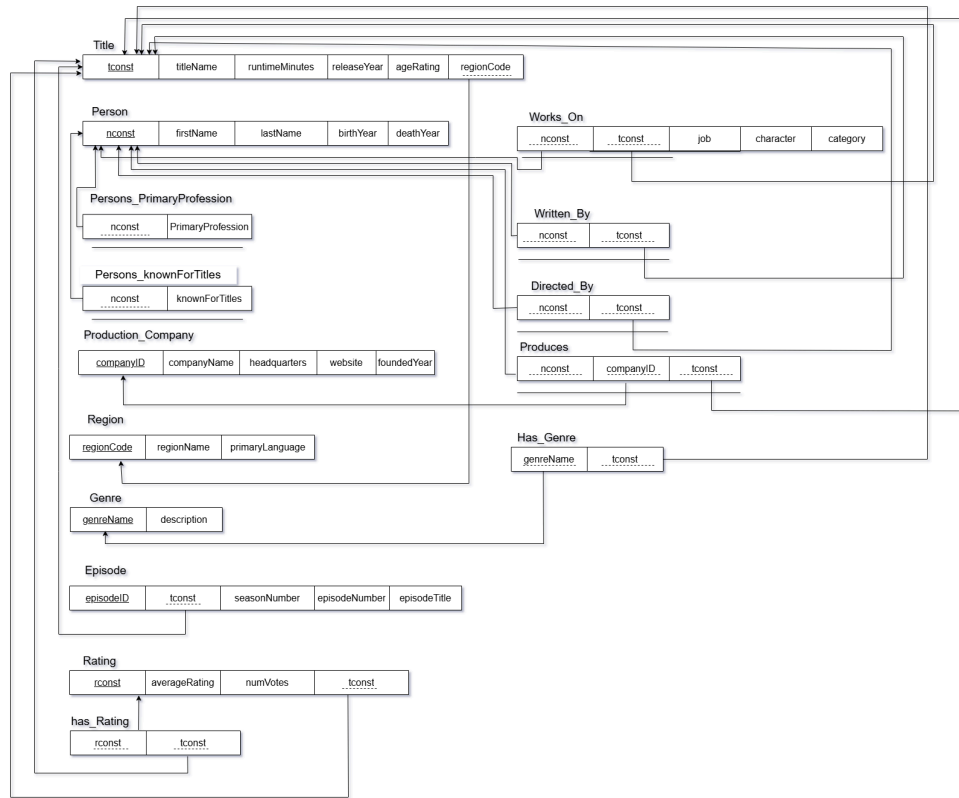


Figure 2: Relational Schema showing entities, attributes, and keys

3.3. Entity Analysis

Entity: Title

Purpose: The central entity of the database. It represents a movie, TV series, or other media production.

Attributes:

- ❖ tconst (Primary Key): A unique identifier for each title.
- ❖ titleName: The name of the title.
- ❖ runtimeMinutes: The duration.
- ❖ releaseYear: The year it was released.
- ❖ ageRating: The intended audience age rating (e.g., PG, R).
- ❖ regionCode (Foreign Key): Links to the Region table, indicating the primary region of release.

Entity: Person

Purpose: Represents an individual involved in the industry (actor, director, producer, etc.).

Attributes:

- ❖ nconst (Primary Key): A unique identifier for each person.
- ❖ firstName & lastName: The person's name.
- ❖ birthYear & deathYear: When a person was born and when they died.

Entity: Persons_Primary/Profession

Purpose: A weak entity to handle a multi-valued attribute. A person can have multiple primary professions.

Attributes:

- ❖ nconst (Foreign Key): References the Person table.
- ❖ Primary/Profession: The profession (e.g., 'actor', 'director').

Entity: Persons_knownForTitles

Purpose: Another weak entity for a multi-valued attribute. Stores the titles a person is best known for.

Attributes:

- ❖ nconst (Foreign Key): References the Person table.
- ❖ knownForTitles (Foreign Key): References Title tconst.

Entity: Production_Company

Purpose: Represents a company that produces titles.

Attributes:

- ❖ companyID (Primary Key): A unique identifier.
- ❖ companyName, headquarters, website, foundedYear: Descriptive details about the company.

Entity: Region

Purpose: Stores information about different regions for release.

Attributes:

- ❖ regionCode (Primary Key): A unique code for the region (e.g., 'US', 'UK').
- ❖ regionName: The full name of the region.
- ❖ primaryLanguage: The main language spoken there.

Entity: Genre

Purpose: Different categories of titles.

Attributes:

- ❖ genreName (Primary Key): The name of the genre (e.g., 'Comedy', 'Sci-Fi').
- ❖ description: A brief explanation of the genre.

Entity: Episode

Purpose: Manages episodes for TV series, which are a specific type of Title.

Attributes:

- ❖ episodeID (Primary Key): A unique identifier for the episode record.
- ❖ tconst (Foreign Key): References the Title table, identifying the series this episode belongs to.
- ❖ seasonNumber: The season's position within the series.
- ❖ episodeNumber: The episode's position within the series.
- ❖ episodeTitle: The specific title of this episode.

Entity: Rating

Purpose: Stores user ratings and vote counts for titles.

Attributes:

- ❖ rconst (Primary Key): A unique identifier for the rating record.
- ❖ averageRating: The calculated average rating.
- ❖ numVotes: The total number of votes received.

IMDb Movie Database Management System

- ❖ tconst (Foreign Key): References the Title table, indicating which title is being rated.

3.4. Relationship Analysis

Title → Region (1:N)

Relationship: A Title is released in one primary Region, but a Region can have many Titles released in it.

Cardinality: One-to-Many (1:N)

Implementation: The regionCode in the Title table acts as a foreign key pointing to the Region table.

Justification: Each title has a single primary region of release, while each region can be the primary release location for multiple titles. The foreign key constraint ensures referential integrity between Title and Region.

Person → Persons_PrimaryProfession (1:N)

Relationship: One Person can have multiple Primary Professions.

Cardinality: One-to-Many (1:N)

Implementation: A separate Persons_PrimaryProfession table stores the multi-valued attribute, with nconst as a foreign key referencing Person.

Justification: This is a 1:N relationship implemented to resolve a multi-valued attribute. Instead of having a single profession column in the Person table (which would limit a person to one profession), a separate table is created where each row holds one profession for a person. This allows individuals to have multiple professional roles (e.g., Director, Writer, Producer).

Person → Persons_knownForTitles (1:N)

Relationship: One Person can be known for multiple Titles.

Cardinality: One-to-Many (1:N)

Implementation: A separate Persons_knownForTitles table stores the multi-valued attribute, with nconst as a foreign key referencing Person and tconst as a foreign key referencing Title.

Justification: This is a 1:N relationship resolving a multi-valued attribute (knownForTitles). Similar to the profession relationship, this design allows a person to be associated with multiple titles they are best known for, avoiding the limitation of a single-valued attribute in the Person table.

Title → Episode (1:N)

Relationship: One TV Series (Title) can have many Episodes.

Cardinality: One-to-Many (1:N)

Implementation: The Episode table has a foreign key tconst pointing back to its parent series in the Title table.

Justification: This is a 1:N relationship representing the hierarchical structure of television series. Each episode belongs to exactly one series, while a series contains multiple episodes. The Episode table includes seasonNumber and episodeNumber to maintain proper ordering within the series structure.

Title → Rating (1:1)

Relationship: One Title has exactly one Rating record, and one Rating record belongs to exactly one Title.

Cardinality: One-to-One (1:1)

Implementation: The Rating table has its own primary key (rconst) and a foreign key (tconst) pointing to Title.

Justification: This is a 1:1 relationship where each title has a single aggregated rating based on user votes. The separation into a distinct table allows for better organization of rating-specific attributes (averageRating, numVotes) while maintaining referential integrity with the Title table.

Person ↔ Title (M:N)

Relationship: Multiple People can work on multiple Titles, and multiple Titles can have multiple People working on them.

Cardinality: Many-to-Many (M:N)

Implementation: This core M:N relationship is implemented through multiple relationship tables:

- ❖ Works_On: Connects a Person to a Title and specifies their job, character (if applicable), and category (e.g., Acting, Crew).
- ❖ Written_By: Specialized relationship defining writer credits.
- ❖ Directed_By: Specialized relationship defining director credits.

Justification: A Person can work on many Titles in various capacities, and a Title requires many People to produce (actors, directors, writers, crew). The M:N relationship is resolved through junction tables that also store relationship-specific attributes. The specialized tables (Written_By, Directed_By) provide clear semantics for specific professional roles while Works_On captures more general participation with additional descriptive attributes.

Person ↔ Production_Company ↔ Title (Ternary M:N:N)

Relationship: A Person (producer) can produce many Titles for many Companies; a Company can produce many Titles with many People; and a Title can be produced by many People working with many Companies.

Cardinality: Many-to-Many-to-Many (Ternary Relationship)

Implementation: This is implemented via the Produces table, which contains foreign keys to all three participating entities (nconst, tconst, companyID).

Justification: This is a complex ternary relationship that captures the production context in the entertainment industry. A producer (Person) doesn't work in isolation—they work with specific companies on specific titles. The three-way relationship is necessary because the association between a producer and a company is meaningful only in the context of a particular title. For example, a producer might work with Warner Bros on one film and Universal Pictures on another, making the three-entity relationship essential for accurate representation.

Title ↔ Genre (M:N)

Relationship: A Title can belong to multiple Genres, and a Genre can contain many Titles.

Cardinality: Many-to-Many (M:N)

Implementation: Implemented via the Has_Genre junction table with foreign keys to both Title (tconst) and Genre (genreName).

Justification: This is a classic M:N relationship reflecting the reality that films and TV shows often span multiple genres (e.g., Action-Comedy, Sci-Fi-Drama). Similarly, each genre encompasses numerous titles. The junction table Has_Genre resolves this M:N relationship by creating a separate record for each title-genre pairing, enabling flexible categorization and genre-based querying.

4. Implementation

4.1. DDL Statements

```
--
=====
=====
-- CORE ENTITY TABLES
--
=====
=====

-- REGION
CREATE TABLE Region (
    regionCode      VARCHAR2(10)    PRIMARY KEY,
    regionName      VARCHAR2(100)   NOT NULL,
    primaryLanguage VARCHAR2(50)
);

-- GENRE
CREATE TABLE Genre (
    genreName  VARCHAR2(50)  PRIMARY KEY,
    description VARCHAR2(255)
);

-- PRODUCTION COMPANY
CREATE TABLE Production_Company (
    companyID      VARCHAR2(10)  PRIMARY KEY,
    companyName    VARCHAR2(255) NOT NULL,
    headquarters   VARCHAR2(255),
    website        VARCHAR2(255),
    foundedYear    NUMBER(4)
);

-- PERSON
CREATE TABLE Person (
    nconst      VARCHAR2(10)  PRIMARY KEY,
    firstName   VARCHAR2(100),
    lastName    VARCHAR2(100),
    birthYear   NUMBER(4),
    deathYear   NUMBER(4)
```

IMDb Movie Database Management System

```
);

-- TITLE
CREATE TABLE Title (
    tconst          VARCHAR2(10)  PRIMARY KEY,
    titleName       VARCHAR2(255) NOT NULL,
    runtimeMinutes  NUMBER,
    releaseYear     NUMBER(4),
    ageRating       VARCHAR2(10),
    regionCode      VARCHAR2(10),
    CONSTRAINT fk_title_region
        FOREIGN KEY (regionCode)
        REFERENCES Region(regionCode),
    CONSTRAINT ck_title_runtime
        CHECK (runtimeMinutes IS NULL OR runtimeMinutes >=
0)
);

-- EPISODE
CREATE TABLE Episode (
    episodeID       VARCHAR2(10)  PRIMARY KEY,
    tconst          VARCHAR2(10)  NOT NULL,
    seasonNumber    NUMBER,
    episodeNumber   NUMBER,
    episodeTitle    VARCHAR2(255),
    CONSTRAINT fk_episode_title
        FOREIGN KEY (tconst)
        REFERENCES Title(tconst)
);

-- RATING
CREATE TABLE Rating (
    rconst          VARCHAR2(10)  PRIMARY KEY,
    averageRating   NUMBER(3,1),
    numVotes        NUMBER,
    tconst          VARCHAR2(10)  NOT NULL,
    CONSTRAINT fk_rating_title
        FOREIGN KEY (tconst)
        REFERENCES Title(tconst),
    CONSTRAINT ck_rating_range
```

IMDb Movie Database Management System

```
        CHECK (averageRating BETWEEN 0 AND 10)
    );

--
=====
=====
-- WEAK ENTITY / MULTI-VALUED ATTRIBUTE TABLES
--
=====
=====

-- PERSONS_PRIMARYPROFESSION (multi-valued attribute)
CREATE TABLE Persons_PrimaryProfession (
    nconst          VARCHAR2(10),
    primaryProfession VARCHAR2(100),
    CONSTRAINT pk_person_primary_prof
        PRIMARY KEY (nconst, primaryProfession),
    CONSTRAINT fk_pp_person
        FOREIGN KEY (nconst)
        REFERENCES Person(nconst)
);

-- PERSONS_KNOWNFORTITLES (multi-valued attribute)
CREATE TABLE Persons_knownForTitles (
    nconst VARCHAR2(10),
    tconst VARCHAR2(10),
    CONSTRAINT pk_person_known_titles
        PRIMARY KEY (nconst, tconst),
    CONSTRAINT fk_pkt_person
        FOREIGN KEY (nconst)
        REFERENCES Person(nconst),
    CONSTRAINT fk_pkt_title
        FOREIGN KEY (tconst)
        REFERENCES Title(tconst)
);

--
=====
=====
-- RELATIONSHIP TABLES
```

IMDb Movie Database Management System

```
--
=====

-- WORKS_ON (Person - Title)
CREATE TABLE Works_On (
    nconst    VARCHAR2(10),
    tconst    VARCHAR2(10),
    job        VARCHAR2(100),
    character  VARCHAR2(255),
    category  VARCHAR2(50),
    CONSTRAINT pk_works_on
        PRIMARY KEY (nconst, tconst),
    CONSTRAINT fk_wo_person
        FOREIGN KEY (nconst)
        REFERENCES Person(nconst),
    CONSTRAINT fk_wo_title
        FOREIGN KEY (tconst)
        REFERENCES Title(tconst)
);

-- WRITTEN_BY (Person - Title)
CREATE TABLE Written_By (
    nconst    VARCHAR2(10),
    tconst    VARCHAR2(10),
    CONSTRAINT pk_written_by
        PRIMARY KEY (nconst, tconst),
    CONSTRAINT fk_wb_person
        FOREIGN KEY (nconst)
        REFERENCES Person(nconst),
    CONSTRAINT fk_wb_title
        FOREIGN KEY (tconst)
        REFERENCES Title(tconst)
);

-- DIRECTED_BY (Person - Title)
CREATE TABLE Directed_By (
    nconst    VARCHAR2(10),
    tconst    VARCHAR2(10),
    CONSTRAINT pk_directed_by
```

IMDb Movie Database Management System

```
        PRIMARY KEY (nconst, tconst),
CONSTRAINT fk_db_person
    FOREIGN KEY (nconst)
    REFERENCES Person(nconst),
CONSTRAINT fk_db_title
    FOREIGN KEY (tconst)
    REFERENCES Title(tconst)
);

-- PRODUCES (Person - Production_Company - Title)
CREATE TABLE Produces (
    nconst    VARCHAR2(10),
    companyID VARCHAR2(10),
    tconst    VARCHAR2(10),
CONSTRAINT pk_produces
    PRIMARY KEY (nconst, companyID, tconst),
CONSTRAINT fk_prod_person
    FOREIGN KEY (nconst)
    REFERENCES Person(nconst),
CONSTRAINT fk_prod_company
    FOREIGN KEY (companyID)
    REFERENCES Production_Company(companyID),
CONSTRAINT fk_prod_title
    FOREIGN KEY (tconst)
    REFERENCES Title(tconst)
);

-- HAS_GENRE (Title - Genre)
CREATE TABLE Has_Genre (
    genreName VARCHAR2(50),
    tconst    VARCHAR2(10),
CONSTRAINT pk_has_genre
    PRIMARY KEY (genreName, tconst),
CONSTRAINT fk_hg_genre
    FOREIGN KEY (genreName)
    REFERENCES Genre(genreName),
CONSTRAINT fk_hg_title
    FOREIGN KEY (tconst)
    REFERENCES Title(tconst)
);
```

IMDb Movie Database Management System

```
-- HAS_RATING (Title - Rating)
CREATE TABLE Has_Rating (
    rconst VARCHAR2(10),
    tconst VARCHAR2(10),
    CONSTRAINT pk_has_rating
        PRIMARY KEY (rconst, tconst),
    CONSTRAINT fk_hr_rating
        FOREIGN KEY (rconst)
        REFERENCES Rating(rconst),
    CONSTRAINT fk_hr_title
        FOREIGN KEY (tconst)
        REFERENCES Title(tconst)
);

--
=====
=====
-- TRIGGERS
--
=====
=====

-- TRIGGER: Automatically inserts into Has_Rating when a
new Rating is inserted
CREATE OR REPLACE TRIGGER trg_insert_has_rating
AFTER INSERT ON Rating
FOR EACH ROW
BEGIN
    INSERT INTO Has_Rating (rconst, tconst)
    VALUES (:NEW.rconst, :NEW.tconst);
END;
/
```


4.2. Data Population

```
--
=====

=====
-- REGION DATA
--
=====
=====
INSERT INTO Region (regionCode, regionName,
primaryLanguage)
VALUES ('US', 'United States', 'English');

INSERT INTO Region (regionCode, regionName,
primaryLanguage)
VALUES ('UAE', 'United Arab Emirates', 'Arabic');

INSERT INTO Region (regionCode, regionName,
primaryLanguage)
VALUES ('KR', 'South Korea', 'Korean');

INSERT INTO Region (regionCode, regionName,
primaryLanguage)
VALUES ('TR', 'Turkey', 'Turkish');

COMMIT;

--
=====

=====
-- GENRE DATA
--
=====
=====
INSERT INTO Genre (genreName, description)
VALUES ('Action', 'High-energy films with physical stunts
and chase sequences');

INSERT INTO Genre (genreName, description)
VALUES ('Drama', 'Serious narrative focusing on character
development');
```

IMDb Movie Database Management System

```
INSERT INTO Genre (genreName, description)
VALUES ('Comedy', 'Entertainment designed to make the
audience laugh');

INSERT INTO Genre (genreName, description)
VALUES ('Sci-Fi', 'Science fiction exploring futuristic
concepts');

COMMIT;

--
=====
=====
-- PRODUCTION COMPANY DATA
--
=====
=====
INSERT INTO Production_Company (companyID, companyName,
headquarters, website, foundedYear)
VALUES ('PC001', 'Warner Bros', 'Burbank, California',
'www.warnerbros.com', 1923);

INSERT INTO Production_Company (companyID, companyName,
headquarters, website, foundedYear)
VALUES ('PC002', 'Universal Pictures', 'Universal City,
California', 'www.universalpictures.com', 1912);

INSERT INTO Production_Company (companyID, companyName,
headquarters, website, foundedYear)
VALUES ('PC003', 'Netflix Studios', 'Los Gatos,
California', 'www.netflix.com', 1997);

INSERT INTO Production_Company (companyID, companyName,
headquarters, website, foundedYear)
VALUES ('PC004', 'A24', 'New York City, New York',
'www.a24films.com', 2012);

INSERT INTO Production_Company (companyID, companyName,
headquarters, website, foundedYear)
```

IMDb Movie Database Management System

```
VALUES ('PC005', 'Legendary Entertainment', 'Burbank,
California', 'www.legendary.com', 2000);

COMMIT;

--
=====
=====
-- PERSON DATA
--
=====
=====
INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM001', 'Christopher', 'Nolan', 1970, NULL);

INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM002', 'Cillian', 'Murphy', 1976, NULL);

INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM003', 'Greta', 'Gerwig', 1983, NULL);

INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM004', 'Margot', 'Robbie', 1990, NULL);

INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM005', 'Pedro', 'Pascal', 1975, NULL);

INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM006', 'Emily', 'Blunt', 1983, NULL);

INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM007', 'Ryan', 'Gosling', 1980, NULL);
```

IMDb Movie Database Management System

```
INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM008', 'Denis', 'Villeneuve', 1967, NULL);
```

```
INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM009', 'Timothée', 'Chalamet', 1995, NULL);
```

```
INSERT INTO Person (nconst, firstName, lastName, birthYear,
deathYear)
VALUES ('NM010', 'Zendaya', 'Coleman', 1996, NULL);
```

```
COMMIT;
```

```
--
=====
=====
-- PERSONS_PRIMARYPROFESSION DATA
--
```

```
=====
=====
-- Christopher Nolan
INSERT INTO Persons_PrimaryProfession (nconst,
primaryProfession)
VALUES ('NM001', 'Director');
```

```
INSERT INTO Persons_PrimaryProfession (nconst,
primaryProfession)
VALUES ('NM001', 'Writer');
```

```
-- Cillian Murphy
INSERT INTO Persons_PrimaryProfession (nconst,
primaryProfession)
VALUES ('NM002', 'Actor');
```

```
-- Greta Gerwig
INSERT INTO Persons_PrimaryProfession (nconst,
primaryProfession)
VALUES ('NM003', 'Director');
```

IMDb Movie Database Management System

```
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM003', 'Writer');
```

```
-- Margot Robbie  
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM004', 'Actor');
```

```
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM004', 'Producer');
```

```
-- Pedro Pascal  
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM005', 'Actor');
```

```
-- Emily Blunt  
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM006', 'Actor');
```

```
-- Ryan Gosling  
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM007', 'Actor');
```

```
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM007', 'Producer');
```

```
-- Denis Villeneuve  
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM008', 'Director');
```

```
INSERT INTO Persons_PrimaryProfession (nconst,  
primaryProfession)  
VALUES ('NM008', 'Writer');
```

IMDb Movie Database Management System

```
-- Timothée Chalamet
INSERT INTO Persons_PrimaryProfession (nconst,
primaryProfession)
VALUES ('NM009', 'Actor');

-- Zendaya
INSERT INTO Persons_PrimaryProfession (nconst,
primaryProfession)
VALUES ('NM010', 'Actor');

COMMIT;

--
=====
=====
-- TITLE DATA
--
=====
=====
INSERT INTO Title (tconst, titleName, runtimeMinutes,
releaseYear, ageRating, regionCode)
VALUES ('TT001', 'Oppenheimer', 180, 2023, 'R', 'US');

INSERT INTO Title (tconst, titleName, runtimeMinutes,
releaseYear, ageRating, regionCode)
VALUES ('TT002', 'Barbie', 114, 2023, 'PG-13', 'US');

INSERT INTO Title (tconst, titleName, runtimeMinutes,
releaseYear, ageRating, regionCode)
VALUES ('TT003', 'Squid Game', 60, 2021, 'TV-MA', 'KR');

INSERT INTO Title (tconst, titleName, runtimeMinutes,
releaseYear, ageRating, regionCode)
VALUES ('TT004', 'The Last of Us', 55, 2023, 'TV-MA',
'US');

INSERT INTO Title (tconst, titleName, runtimeMinutes,
releaseYear, ageRating, regionCode)
```

IMDb Movie Database Management System

```
VALUES ('TT005', 'Dune: Part Two', 166, 2024, 'PG-13',  
'US');
```

```
INSERT INTO Title (tconst, titleName, runtimeMinutes,  
releaseYear, ageRating, regionCode)  
VALUES ('TT006', 'Inception', 148, 2010, 'PG-13', 'US');
```

```
INSERT INTO Title (tconst, titleName, runtimeMinutes,  
releaseYear, ageRating, regionCode)  
VALUES ('TT007', 'The Dark Knight', 152, 2008, 'PG-13',  
'US');
```

```
INSERT INTO Title (tconst, titleName, runtimeMinutes,  
releaseYear, ageRating, regionCode)  
VALUES ('TT008', 'Stranger Things', 50, 2016, 'TV-14',  
'US');
```

```
COMMIT;
```

```
--  
=====
```

Rating	Movie	Average Rating	Num Votes
R001	TT001	8.5	650000
R002	TT002	7.0	450000
R003	TT003	8.0	580000

```
-- RATING DATA  
--  
=====
```

```
INSERT INTO Rating (rconst, averageRating, numVotes,  
tconst)  
VALUES ('R001', 8.5, 650000, 'TT001');
```

```
INSERT INTO Rating (rconst, averageRating, numVotes,  
tconst)  
VALUES ('R002', 7.0, 450000, 'TT002');
```

```
INSERT INTO Rating (rconst, averageRating, numVotes,  
tconst)  
VALUES ('R003', 8.0, 580000, 'TT003');
```

```
INSERT INTO Rating (rconst, averageRating, numVotes,  
tconst)
```

IMDb Movie Database Management System

```
VALUES ('R004', 8.8, 720000, 'TT004');

INSERT INTO Rating (rconst, averageRating, numVotes,
tconst)
VALUES ('R005', 8.7, 520000, 'TT005');

INSERT INTO Rating (rconst, averageRating, numVotes,
tconst)
VALUES ('R006', 8.8, 2400000, 'TT006');

INSERT INTO Rating (rconst, averageRating, numVotes,
tconst)
VALUES ('R007', 9.0, 2800000, 'TT007');

INSERT INTO Rating (rconst, averageRating, numVotes,
tconst)
VALUES ('R008', 8.7, 1200000, 'TT008');

COMMIT;

--
=====
=====
-- EPISODE DATA (TV Series Only)
--
=====
=====
-- Squid Game Episodes
INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP001', 'TT003', 1, 1, 'Red Light, Green Light');

INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP002', 'TT003', 1, 2, 'Hell');

INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP003', 'TT003', 1, 3, 'The Man with the
Umbrella');
```


IMDb Movie Database Management System

```
-- The Last of Us Episodes
INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP004', 'TT004', 1, 1, 'When You''re Lost in the
Darkness');

INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP005', 'TT004', 1, 2, 'Infected');

-- Stranger Things Episodes
INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP006', 'TT008', 1, 1, 'Chapter One: The Vanishing
of Will Byers');

INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP007', 'TT008', 1, 2, 'Chapter Two: The Weirdo on
Maple Street');

INSERT INTO Episode (episodeID, tconst, seasonNumber,
episodeNumber, episodeTitle)
VALUES ('EP008', 'TT008', 2, 1, 'Chapter One: MADMAX');

COMMIT;

--
=====
=====
-- PERSONS_KNOWNFORTITLES DATA
--
=====
=====
-- Christopher Nolan
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM001', 'TT001');

INSERT INTO Persons_knownForTitles (nconst, tconst)
```

IMDb Movie Database Management System

```
VALUES ('NM001', 'TT006');

INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM001', 'TT007');

-- Cillian Murphy
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM002', 'TT001');

-- Greta Gerwig
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM003', 'TT002');

-- Margot Robbie
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM004', 'TT002');

-- Pedro Pascal
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM005', 'TT004');

-- Emily Blunt
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM006', 'TT001');

-- Ryan Gosling
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM007', 'TT002');

-- Denis Villeneuve
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM008', 'TT005');

-- Timothée Chalamet
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM009', 'TT005');

-- Zendaya
INSERT INTO Persons_knownForTitles (nconst, tconst)
VALUES ('NM010', 'TT005');
```

IMDb Movie Database Management System

```
COMMIT;

--
=====
=====
-- WORKS_ON DATA (Acting roles with characters)
--
=====
=====
INSERT INTO Works_On (nconst, tconst, job, character,
category)
VALUES ('NM002', 'TT001', 'Lead Actor', 'J. Robert
Oppenheimer', 'Acting');

INSERT INTO Works_On (nconst, tconst, job, character,
category)
VALUES ('NM004', 'TT002', 'Lead Actor', 'Barbie',
'Acting');

INSERT INTO Works_On (nconst, tconst, job, character,
category)
VALUES ('NM005', 'TT004', 'Lead Actor', 'Joel Miller',
'Acting');

INSERT INTO Works_On (nconst, tconst, job, character,
category)
VALUES ('NM006', 'TT001', 'Supporting Actor', 'Kitty
Oppenheimer', 'Acting');

INSERT INTO Works_On (nconst, tconst, job, character,
category)
VALUES ('NM007', 'TT002', 'Lead Actor', 'Ken', 'Acting');

INSERT INTO Works_On (nconst, tconst, job, character,
category)
VALUES ('NM009', 'TT005', 'Lead Actor', 'Paul Atreides',
'Acting');
```

IMDb Movie Database Management System

```
INSERT INTO Works_On (nconst, tconst, job, character,
category)
VALUES ('NM010', 'TT005', 'Lead Actor', 'Chani', 'Acting');

COMMIT;

--
=====
=====
-- DIRECTED_BY DATA
--
=====
=====
-- Christopher Nolan Films
INSERT INTO Directed_By (nconst, tconst)
VALUES ('NM001', 'TT001');

INSERT INTO Directed_By (nconst, tconst)
VALUES ('NM001', 'TT006');

INSERT INTO Directed_By (nconst, tconst)
VALUES ('NM001', 'TT007');

-- Greta Gerwig Films
INSERT INTO Directed_By (nconst, tconst)
VALUES ('NM003', 'TT002');

-- Denis Villeneuve Films
INSERT INTO Directed_By (nconst, tconst)
VALUES ('NM008', 'TT005');

COMMIT;

--
=====
=====
-- WRITTEN_BY DATA
--
=====
=====
```

IMDb Movie Database Management System

```
-- Christopher Nolan Films
INSERT INTO Written_By (nconst, tconst)
VALUES ('NM001', 'TT001');

INSERT INTO Written_By (nconst, tconst)
VALUES ('NM001', 'TT006');

INSERT INTO Written_By (nconst, tconst)
VALUES ('NM001', 'TT007');

-- Greta Gerwig Films
INSERT INTO Written_By (nconst, tconst)
VALUES ('NM003', 'TT002');

-- Denis Villeneuve Films
INSERT INTO Written_By (nconst, tconst)
VALUES ('NM008', 'TT005');

COMMIT;

--
=====
=====
-- PRODUCES DATA (Ternary: Person - Company - Title)
--
=====
=====
-- Christopher Nolan Productions
INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM001', 'PC001', 'TT001');

INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM001', 'PC001', 'TT006');

INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM001', 'PC001', 'TT007');

-- Greta Gerwig Productions
INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM003', 'PC001', 'TT002');
```

IMDb Movie Database Management System

```
-- Margot Robbie Productions
INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM004', 'PC004', 'TT002');

-- Denis Villeneuve Productions
INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM008', 'PC001', 'TT005');

INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM008', 'PC005', 'TT005');

-- Ryan Gosling Productions
INSERT INTO Produces (nconst, companyID, tconst)
VALUES ('NM007', 'PC003', 'TT008');

COMMIT;

--
=====
=====
-- HAS_GENRE DATA (Title - Genre relationships)
--
=====
=====
-- Oppenheimer
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Drama', 'TT001');

INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Action', 'TT001');

-- Barbie
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Comedy', 'TT002');

-- Squid Game
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Drama', 'TT003');
```

IMDb Movie Database Management System

```
-- The Last of Us
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Action', 'TT004');

INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Drama', 'TT004');

-- Dune: Part Two
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Sci-Fi', 'TT005');

INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Action', 'TT005');

-- Inception
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Sci-Fi', 'TT006');

INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Action', 'TT006');

-- The Dark Knight
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Action', 'TT007');

INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Drama', 'TT007');

-- Stranger Things
INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Sci-Fi', 'TT008');

INSERT INTO Has_Genre (genreName, tconst)
VALUES ('Drama', 'TT008');

COMMIT;
```

5. Query Demonstration

5.1. Query 1: Find all titles with their ratings and genres

```

SELECT
    t.titleName,
    t.releaseYear,
    r.averageRating,
    r.numVotes,
    g.genreName
FROM Title t
JOIN Rating r ON t.tconst = r.tconst
JOIN Has_Genre hg ON t.tconst = hg.tconst
JOIN Genre g ON hg.genreName = g.genreName
WHERE r.averageRating >= 8.0
ORDER BY r.averageRating DESC, t.releaseYear DESC;

```

	TITLENAME	RELEASEYEAR	AVERAGERATING	NUMVOTES	GENRENAME
1	The Dark Knight	2008	9	2800000	Action
2	The Dark Knight	2008	9	2800000	Drama
3	The Last of Us	2023	8.8	720000	Drama
4	The Last of Us	2023	8.8	720000	Action
5	Inception	2010	8.8	2400000	Action
6	Inception	2010	8.8	2400000	Sci-Fi
7	Dune: Part Two	2024	8.7	520000	Action
8	Dune: Part Two	2024	8.7	520000	Sci-Fi
9	Stranger Things	2016	8.7	1200000	Drama
10	Stranger Things	2016	8.7	1200000	Sci-Fi
11	Oppenheimer	2023	8.5	650000	Action
12	Oppenheimer	2023	8.5	650000	Drama
13	Squid Game	2021	8	580000	Drama

Figure 3

5.2. Query 2: Count the number of titles directed by each director

```

SELECT
    p.firstName || ' ' || p.lastName AS directorName,
    p.birthYear,
    COUNT(db.tconst) AS numberOfFilms
FROM Person p

```


IMDb Movie Database Management System

```
JOIN Directed_By db ON p.nconst = db.nconst
GROUP BY p.nconst, p.firstName, p.lastName, p.birthYear
HAVING COUNT(db.tconst) >= 2
ORDER BY numberOfFilms DESC;
```

	DIRECTORNAME	BIRTHYEAR	NUMBEROFFILMS
1	Christopher Nolan	1970	3

Figure 4

5.3. Query 3: Find actors and the characters they played in specific titles

```
SELECT
    t.titleName,
    p.firstName || ' ' || p.lastName AS actorName,
    wo.character AS characterPlayed,
    wo.job,
    t.releaseYear
FROM Title t
JOIN Works_On wo ON t.tconst = wo.tconst
JOIN Person p ON wo.nconst = p.nconst
WHERE wo.category = 'Acting'
ORDER BY t.releaseYear DESC, t.titleName;
```

	TITLENAME	ACTORNAME	CHARACTERPLAYED	JOB	RELEASEYEAR
1	Dune: Part Two	Timothée Chalamet	Paul Atreides	Lead Actor	2024
2	Dune: Part Two	Zendaya Coleman	Chani	Lead Actor	2024
3	Barbie	Ryan Gosling	Ken	Lead Actor	2023
4	Barbie	Margot Robbie	Barbie	Lead Actor	2023
5	Oppenheimer	Cillian Murphy	J. Robert Oppenheimer	Lead Actor	2023
6	Oppenheimer	Emily Blunt	Kitty Oppenheimer	Supporting Actor	2023
7	The Last of Us	Pedro Pascal	Joel Miller	Lead Actor	2023

Figure 5

5.4. Query 4: Average rating and total votes by genre

```
SELECT
    g.genreName,
    COUNT(DISTINCT t.tconst) AS numberOfTitles,
```

IMDb Movie Database Management System

```
ROUND(AVG(r.averageRating), 2) AS avgGenreRating,  
SUM(r.numVotes) AS totalVotes  
FROM Genre g  
JOIN Has_Genre hg ON g.genreName = hg.genreName  
JOIN Title t ON hg.tconst = t.tconst  
JOIN Rating r ON t.tconst = r.tconst  
GROUP BY g.genreName  
ORDER BY avgGenreRating DESC;
```

	GENRENAME	NUMBEROFTITLES	AVGGENRERATING	TOTALVOTES
1	Action	5	8.76	7090000
2	Sci-Fi	3	8.73	4120000
3	Drama	5	8.6	5950000
4	Comedy	1	7	450000

Figure 6

6. Conclusion

This project successfully designed and implemented a comprehensive relational database management system for IMDb movie and television data. The system demonstrates a thorough application of fundamental database concepts, including entity-relationship modeling, normalization principles, and SQL implementation. The database architecture effectively captures the complex relationships inherent in the entertainment industry. Through careful design, we implemented a normalized schema that handles multi-valued attributes, ternary relationships, and various cardinalities while maintaining referential integrity. The use of junction tables for many-to-many relationships and separate tables for multi-valued attributes ensures data consistency and eliminates redundancy.

7. References

IMDb Datasets. (n.d.). *IMDb Non-Commercial Datasets*. IMDb Developer. Retrieved November 26, 2025, from <https://developer.imdb.com/non-commercial-datasets/>

IMDb. (n.d.). *IMDb Datasets*. Retrieved November 26, 2025, from <https://datasets.imdbws.com/>